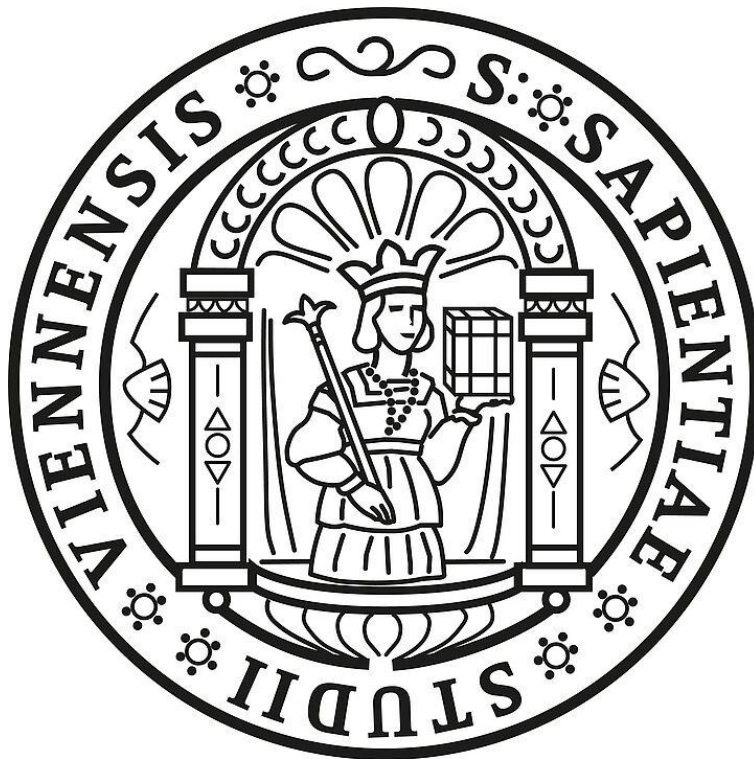


Secure IoT Monitoring System Report

Performance Evaluation and Analysis



Student: Gentijan Bakiu

Supervisor: Ass.-Prof. Dott. mag. Dr. Atakan Aral

Contents

1. Introduction	3
1.1 Motivation	3
1.2 Design Rationale	3
2. System Architecture Recap	4
3. Monitoring System Setup.....	6
3.1. Prometheus as Central Metrics Engine.....	6
3.2. Node Exporter for Host Metrics.....	6
3.3. Telegraf for Container and Custom Metrics	7
3.4. Custom Instrumentation	7
3.5. Grafana For Visualization and Dashboards	7
4. Evaluation Methodology	10
4.1. Experimental Setup	10
4.2. Load Scenarios and Duration	10
5. Performance Analysis	11
5.2. Resource Usage and Container Overhead	11
5.3. Monitoring Overhead	11
5.4. Core Components Stability	12
5.5 Snapshots of the Dashboards	12
6. Future Work.....	15
6. Conclusion.....	16
7. References.....	17

1. Introduction

1.1 Motivation

This project is based on the concept of secure and scalable monitoring for critical infrastructures (CI), and is inspired by the TROCI project (Towards Resilient Operation of Critical Infrastructure) a research initiative that our university is actively involved in. TROCI focuses on improving the resilience of critical infrastructures through new designs of instrumentation and control (I&C) that reduce attack surfaces and enable more localized and smarter data handling.

While my system does not implement the full scope of TROCI's vision it still explores a related problem on a smaller scale which is: how well can a centralized IoT monitoring setup handle increasing amounts of data under realistic load? This was the main question behind the second phase of my work and the reason that I focused on performance testing and architectural stress evaluation.

This phase builds directly on the system that I developed during my "Practical 1" course, where I implemented a containerized IoT infrastructure that simulates secure communication between multiple devices and backend services. The system includes publisher containers that send temperature and humidity data via MQTT, a subscriber that processes the incoming messages, and a database and dashboard layer for storage and visualization. For "Practical 2" I extended this setup by integrating a full monitoring stack using Prometheus, Telegraf and also Node Exporter in order to track system and container-level performance, this allowed me to systematically scale the number of simulated devices and observe how the system behaves as the load increases both in terms of stability and resource usage.

1.2 Design Rationale

The system is designed around containerized microservices to allow for flexible deployment, easy replication of devices and isolated testing of components. Docker Compose was used to orchestrate the setup and simulate large numbers of IoT devices without requiring additional hardware.

Each simulated device runs a lightweight container publishing sensor data via the MQTT protocol to a central broker (Mosquitto), this design was chosen for its reliability and lightweight footprint which makes it well-suited for real-world IoT scenarios where devices often operate with limited resources, such as low power, minimal memory or unreliable network connections.

The data then is processed by a Python based subscriber service, which parses the incoming MQTT messages and writes them to InfluxDB as an efficient database for handling high-frequency time-series data that makes it very well suited for storing continuous sensor readings in IoT systems. Grafana is used as the visualization layer because it offers flexible dashboards and real-time monitoring of sensor values and as well, a very straightforward integration with InfluxDB.

To support the goals of performance analysis and system evaluation, I added a dedicated monitoring stack which includes:

- **Prometheus** for centralized metric scraping due to its strong integration with Docker and native support for time-series monitoring.
- **Telegraf** for collecting detailed container-level statistics like CPU, memory and network usage to understand how each service behaves under different loads.
- **Node Exporter** for collecting low-level host system metrics to help us track overall CPU load, memory consumption and overall system health.

By having this design we are able to have real-time monitoring and also historical performance analysis, which makes it easier to detect bottlenecks and understand system behavior as the number of simulated devices increases.

2. System Architecture Recap

The core system as I mentioned was already developed in my P1 course, where I implemented a containerized IoT data pipeline using Docker Compose [\[7\]](#). Below I will introduce a quick summary of the main components to give some context on the performance testing and monitoring that I implemented in P2.

- **MQTT Publisher Devices:** I use lightweight Python scripts inside Docker containers to simulate IoT sensors, where each container generates random temperature and humidity values and publishes them to a shared MQTT topic every few seconds, and the number of devices can be scaled easily by using “**docker-compose up --scale publisher_device=100**”
- **MQTT Broker (Mosquitto):** I use Mosquitto as the central communication hub, it receives all published messages and makes them available to my subscriber, and during my tests it turned out that it could handle high client counts and worked reliably even in the heaviest load during my performance testing.

- **Subscriber Service:** This is a Python-based container that subscribes to the topic and processes the incoming data from the MQTT, It parses each JSON payload and it writes it directly into my InfluxDB [3].
- **InfluxDB:** I use InfluxDB to store all sensor data as time-series records where each reading is tagged with a device ID and timestamps making it easy to query and analyze trends over time. During my P1 I used the cloud version of InfluxDB because I had trouble with token persistence over restarts, but during stress tests it wasn't possible to work with it because of the high number of writes and reads, so I decided to switch to containerized InfluxDB where I fixed my persistence issues and had no problems over high loads.
- **Grafana:** For visualization, I set up Grafana dashboards connected to InfluxDB, where I show real-time charts of sensor data and also include alerts that trigger messages into a discord server when values exceed certain thresholds, and later I also use this to display my performance data.

The diagram below provides a high-level overview of this core architecture built during my “Practical 1”, it shows how the simulated publisher containers send sensor data via MQTT to the Mosquitto broker, which then forwards the messages to the subscriber service for processing. The subscriber then sends writes the data into InfluxDB and Grafana queries the database to visualize this data in a real0time dashboard and trigger alerts when certain thresholds are exceeded.

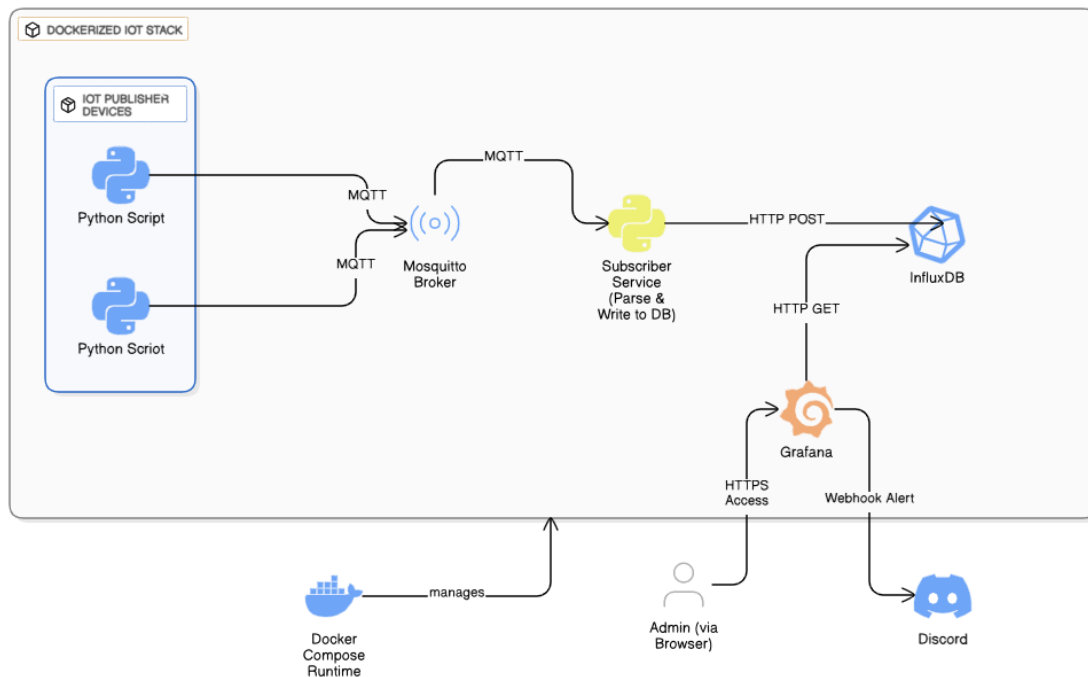


Figure 1: P1 System Architecture Diagram

3. Monitoring System Setup

To evaluate the performance and scalability of my IoT system under increasing load, I extended the existing Docker-based setup with a full monitoring infrastructure which gives me a detailed observation on the resource consumption, message throughput and also how each service responded to heavy loads as the number of IoT devices increased. The goal was to monitor both system-level and container-level resource usage (CPU, memory, network) and identify potential bottlenecks and to verify that the data pipeline worked correctly and the core services could scale with increasing device counts.

3.1. Prometheus as Central Metrics Engine

Prometheus [1] was used to scrape metrics from all the sources that I have defined every few seconds and store them as time-series data, the reason that I chose Prometheus was because it integrates well with Docker, and it is very easy to retrieve and then visualize the data in Grafana since it works natively with it.

I configured Prometheus to collect metrics from:

- **Node Exporter** for host machine stats (CPU, RAM, System Load)
- **Telegraf** for collecting the stats of each container in itself
- **Custom Endpoints** exposed by my subscriber and broker monitor containers.

Unlike InfluxDB or Graphite, Prometheus is better suited for dynamic containerized environments, where it uses a pull model instead of requiring agents to push data, which simplifies the integration process and makes scaling easier. Also I wanted to reserve InfluxDB purely to store sensor data because I didn't want to overload it with the system metrics which would impact its performance.

3.2. Node Exporter for Host Metrics.

To get detailed information on the systems overall resource consumption I deployed Node Exporter [6] as a dedicated lightweight container on the host that reports OS-level metrics To track low-level system resource utilization (such as CPU usage, Memory Load and Disk Activity. It exposes the hosts internal metrics via HTTP in a readable format which makes it easy to integrate with Prometheus.

Also another key reason that I chose Node Exporter is its seamless compatibility with Dockerized environments, and it is widely supported by Grafana which made it easy to setup some prebuilt panels for my dashboard for detailed CPU and RAM usage.

3.3. Telegraf for Container and Custom Metrics

Even though Node-Exporter gave me some very clear data on the systems overall metrics, I couldn't make use of it to gather metrics on each container individually, and because I wanted to test the bottlenecks and stats of each of my services, I introduced **Telegraf** [2] to my system, where it is used to collect fine-grained metrics for each container in the system.

I also tested tools like cAdvisor which also expose container stats, it lacks flexibility, was more difficult to configure and it struggles or becomes very resource heavy at large scales, especially with hundreds of publisher devices.

The way that it works is that it connects to the Docker socket and gathers container-specific stats like CPU usage, Memory usage and as well as Network Traffic data, this made it easy to track how each component (e.g. broker, subscriber, InfluxDB, Grafana and all publisher devices) behaved under different loads. I also configured telegraf to expose these metrics in a Prometheus-compatible format via an HTTP endpoint.

3.4. Custom Instrumentation

To confirm that no messages were lost under load, I needed to do some custom instrumentation on the broker service and the subscriber.

- **Subscriber Metrics:** I modified the subscriber service to expose an HTTP /metric Endpoint using the Prometheus Python client, it reported internal counters such as the total messages received from the **broker** and InfluxDB write successes and errors(if any). I added these metrics because I wanted to make sure that each message that was received by the subscriber was also written in the Database successfully.
- **Broker Monitor Service:** Since the Mosquitto broker does not expose Prometheus metrics natively, I decided to create a lightweight monitoring service just for my broker, this service subscribed to Mosquitto's special \$SYS topics (client connections and message counts) and then translated them into Prometheus-compatible metrics, where with these metrics I can now see how many messages were sent from the publisher devices to the Mosquitto broker for then to compare with the metrics that I get from the subscriber. Also I get the number of connected devices.

3.5. Grafana For Visualization and Dashboards

To visualize all the metrics that were collected by Prometheus I used Grafana [3] because it works very well with it and also since I already had it configured for the Sensor Data it made

sense to use that as well for the Performance Visualizations, I also tried some different methods of plotting the data that I collected cleaned and merged them into one csv file, but it was very difficult to plot that huge amount of data in any way, and that's the part where Grafana shines.

So for this project I created a second Grafana dashboard which includes some prebuilt panels and also panels that I have built myself, and they display:

- **System-level stats:** CPU usage, Memory usage and System Load
- **Container Breakdowns:** CPU, Memory and Network Usage for each container
- **Publisher Stats:** Aggregated usage for all publisher devices
- **Application throughput metrics:** MQTT messages received, messages processed by the subscriber and InfluxDB write count.

Grafana made it very easy for me to visualize all systems and performance metrics in a clear and continuously updating way, unlike static logs or CSV exports and plots, it allowed me to monitor the system live during runtime without stopping or exporting data, and this is crucial I think in real-world scenarios because at each moment you can see what components are at most risk of failing and we can take precautions on that. In this document I will also include some Snapshot links and Screenshots of the dashboard.

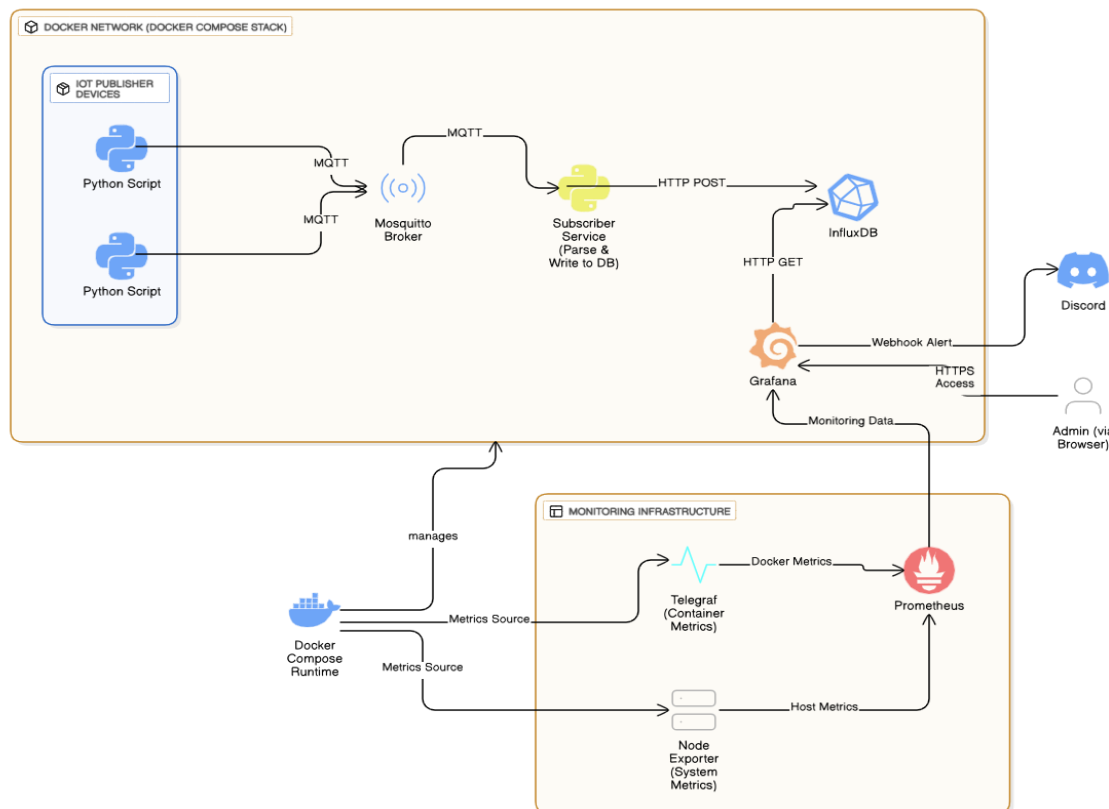
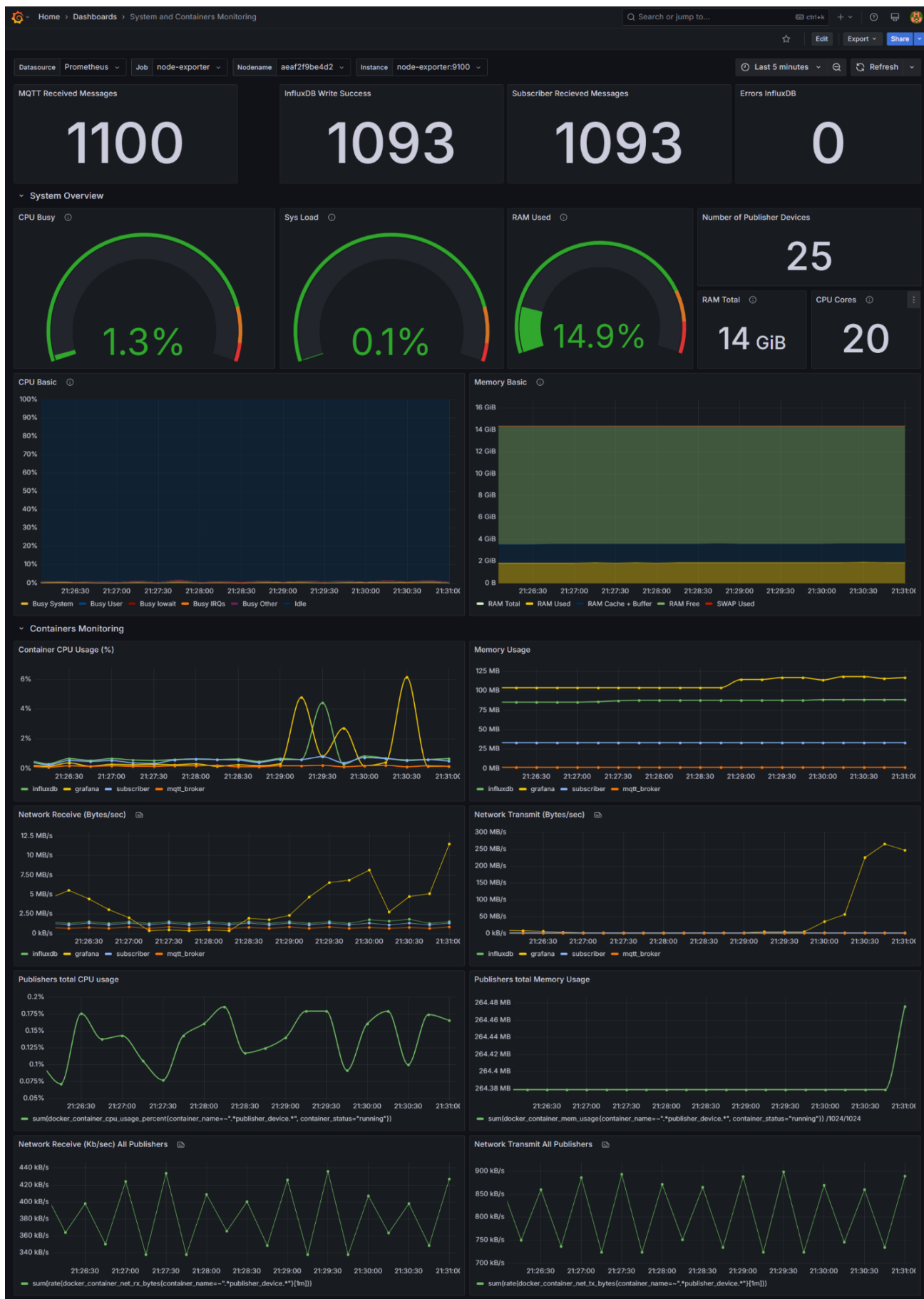


Figure 2: P2 System Architecture Diagram



4. Evaluation Methodology

After setting up my monitoring system I conducted a series of stress tests to evaluate how the system performs under different load levels. The goal was to simulate a growing number of publisher devices and observe CPU, memory, and network behavior and to find the systems limits in the way that is designed, in that way we can figure out ways to improve it.

4.1. Experimental Setup

All the tests were run on a single host machine using Docker Compose, the setup included all core services (broker, subscriber, InfluxDB, Grafana), and the monitoring stack (Prometheus, Node Exporter, Telegraf), and a varying number of publisher containers. The machine had **20 CPU cores** and **14 GB** of **RAM**.

I started each scenario with a clean system and new compose setup to avoid leftover state or memory buildup from previous runs, this way I ensured that all metrics and behaviors were isolated for each specific device count.

I noticed throughout the way that the way I setup my tests, by having all publisher devices run in the same machine as the other services that the heaviest load of the system was creating and maintaining the publisher containers, which in a real-world scenario wouldn't be like that but each device would be on its own environment and only send data through endpoints.

I ran each test for about 5 to 10 minutes, which provided enough time to observe resource usage trends and detect any memory or CPU issues, and before taking measurements, I always allowed a short warm-up period (1–2 minutes) so that all containers could start properly and publishers would be fully connected and publishing.

4.2. Load Scenarios and Duration

To simulate different scales of IoT deployment, I used Docker Composes scaling feature to adjust the number of publisher containers:

- `docker-compose up --scale publisher device=10`
- `docker-compose up --scale publisher device=50`
- `docker-compose up --scale publisher device=100`
- and so on up to 400 containers

I configured each publisher device to send a new sensor reading every **10 seconds**, I chose this rate to simulate a steady but realistic IoT environment that was enough to put the system under pressure but without flooding it immediately. With this setup 100 devices would produce around 600 messages a minute.

5. Performance Analysis

In this section I will explain my analysis and insights on the performance testing of my system where the main goal was to identify performance bottlenecks, and the limits of my system, and during the way I also figured out some interesting factors that caused the most amount of overhead which I have collected through extensive testing with a varying number of simulated IoT publisher devices (10,50,100,200,300,400) that constantly at every 10 seconds send data to my system I have collected detailed metrics from my Grafana dashboards which I will explain here and also provide snapshots of my dashboards at each iteration.

5.2. Resource Usage and Container Overhead

The most significant resource overhead actually came from simulating IoT devices as individual Docker containers, where each container adds memory and CPU overhead due to isolation and process management, so even that this design is clean and works well for smaller scale, as container count grows it causes cumulative system pressure.

- Each Publisher Docker container consumed approximately 10 MB of Memory.
- This at a high scale (e.g. 300+ containers), consumed around 40% of the Memory and overall (70-92%) of Memory was consumed from the whole system
- CPU usage also increased due to Docker context-switching overhead, with 12.4% CPU busy and load averages spiking dramatically.
- At 400 Devices the system began actively using swap memory which indicated memory saturation and performance degradation

5.3. Monitoring Overhead

The monitoring stack especially **Telegraf** and **Prometheus** introduced significant overhead, especially at high container counts, and this was probably because Telegraf would try to scrape metrics from hundreds of publisher containers which would overload the system with frequent scraping, causing CPU spikes and high memory use.

- Telegrafs frequent metric scraping of hundreds of containers resulted in increased CPU spikes and high memory consumption
- Prometheus internal data storage and metric management also became increasingly resource-intensive

- And the gaps that we can see at the Grafana panels at around 400 devices, is these monitoring tools crashing.

5.4. Core Components Stability

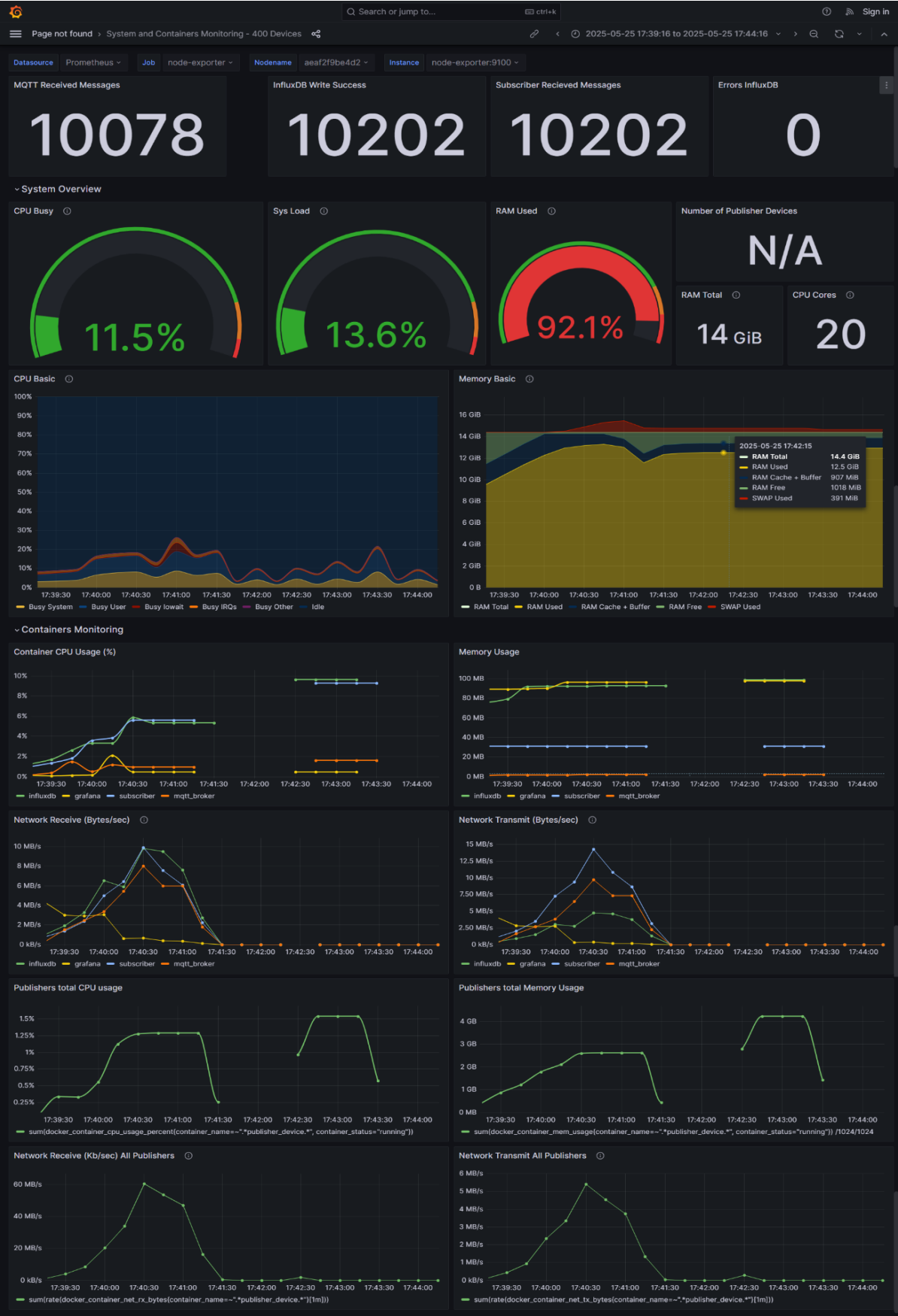
Despite the overheads above, the core IoT components which were the point of this task showed consistent and stable performance, and this was likely due to how lightweight and focused each core service is. They don't do more than they need to do, and since they don't compete for heavy resources like the monitoring tools did, they were able to stay consistent even when the system was under stress.

- **MQTT Broker (Mosquitto):** Remained stable, with minimal CPU (<2%) and low memory usage during all phases of the stress tests.
- **Subscriber Service:** Handled message throughput efficiently and maintained a stable memory usage of less than 100 MB and CPU usage constantly under 10%
- **InfluxDB:** Also demonstrated a very efficient performance with minimal CPU and Memory Usage
- **Grafana:** This service had in specific intervals (when retrieving data) the highest amount of usage from the other services with CPU usage rising up to 15% and but memory remained stable.
- **Message Throughput:** The system consistently maintained an end to end delivery of messages across all the stress tests and even at the highest load the number of MQTT messages received by the broker matched the messages written to InfluxDB by the subscriber.

5.5 Snapshots of the Dashboards

- **10 Devices Snapshot:** System fully stable under light load. Very low CPU and RAM usage.
<https://snapshots.raintank.io/dashboard/snapshot/0Y7XAsJdUUFu6Cfwkk2Hd5Y2YQQYe81g?orgId=0>
- **50 Devices Snapshot:** System handled moderate load well. CPU usage stayed low, RAM usage increased slightly.
<https://snapshots.raintank.io/dashboard/snapshot/FbB9UtMQsrc0EULugYPrrPox1oJZ8YRV?orgId=0>
- **100 Devices Snapshot:** System stayed stable with RAM usage near 30%. Network activity rose, but all components handled the load without data loss or issues.
<https://snapshots.raintank.io/dashboard/snapshot/5olGI0ajJMqMd3XBiBhvQ5CpfXsW3zXoJ>

- **200 Devices Snapshot:** The system handled 200 devices with moderate stress. CPU and memory usage grew notably (RAM ~52%), but no errors or data loss occurred.
<https://snapshots.raintank.io/dashboard/snapshot/TLn164Xj3uPvXuDC4BcwQauyXcOzARSj>
- **300 Devices Snapshot:** RAM usage rises to ~76%, showing the system is nearing limits. CPU and network remain stable. Overall, performance is still solid but close to saturation.
<https://snapshots.raintank.io/dashboard/snapshot/lyga8PT2CgQthWT5hOeqCOT0klbUWN1g?orgId=0>
- **400 Devices Snapshot:** The system reaches critical resource usage, RAM usage spikes to 92% and swap is triggered. CPU and system load increase significantly, suggesting saturation. Telegraf crashes after a certain point
<https://snapshots.raintank.io/dashboard/snapshot/7sscvSSBQozDf0xuOodbeel93vJUrc5m?orgId=0>



6. Future Work

From Simplified Monitoring to Real-World Demands

The current system that I built monitors only basic metrics like (temperature and humidity), while on the other hand real-world critical infrastructures such as flood warning systems, smart grids and industrial monitoring deal with far more complex data that come from diverse sensors like cameras, pressure meters and more. These systems must process high frequency and high volume data streams and react quickly to potentially hazardous conditions

Limitations of the Single-Host Deployment

During testing my system was deployed on a single host running all core services and hundreds of containerized publisher devices, which was manageable at small scale, but the setup showed major limitations when I tried to monitor hundreds of publisher containers on a single node and this led to performance bottlenecks especially from the Monitoring Infrastructure. These observations highlight the need to change the architecture to a more distributed model, where the load is balanced across multiple machines and services are decoupled.

Edge Computing as a Future Direction

To overcome the limitations that I discussed above as well as support for Real-world deployments, my main focus will be on transition the system towards an edge computing model. Edge computing brings computation and decision making closer to the data source and this makes distributed processing directly at or near the sensors possible. This would enable edge nodes to locally analyze sensor data, detect anomalies in real time and only forward essential insights or alerts to the central system without overwhelming it.

In our use case this would mean that instead of sending every raw temperature or humidity value to a central database, each edge node could monitor a group of sensors, detect abnormal patterns such as spike in temperature, drop in humidity, and many more depending on the system that we decide to build, and this would allow for alerts to be triggered immediately, or actions to be taken even when internet connection or the central system is down, which would prevent attacks on the infrastructure to have an effect on the sensors.

This shift in architecture would bring several key benefits specific to the needs of our system as it scales:

- **Scalability:** Each edge node can manage its local sensors independently which makes it much more practical to monitor hundreds or thousands of devices without central overload
- **Faster Alerts:** Edge nodes can raise alerts instantly based on local thresholds or machine learning without waiting for a central decision (which in our system turned out to be slow when managing alerts from hundreds of devices)
- **Network Efficiency:** Only important summaries, events or alerts are sent over the network reducing bandwidth usage which is important for deployments in areas with limited connectivity.
- **Resilience:** Even if the mains server goes down the edge nodes can still continue functioning and responding ensuring that the system remains reliable
- **Security:** Distributing the system across edge nodes reduces the risk of a single point of failure, and in critical infrastructures is one of the most important factors because this limits the impact of targeted cyberattacks, and even if one node fails the others remain functional and they still can continue issuing alerts or taking actions locally and ensure the safety of the whole infrastructure.

6. Conclusion

In this project I demonstrated the design, deployment and performance evaluation of a scalable and secure IoT monitoring system using containerized components and open source tools. By simulating up to 400 IoT publisher devices I was able to assess how the system behaves under various load levels and identify key performance bottlenecks and stability points.

The results that I got clearly showed that the core components such as: Mosquitto, Subscriber, InfluxDB and Grafana remained stable and performed very well even under the highest loads, but the overhead was introduced by Docker containerization of each device and the intensive monitoring setup especially Telegraf where we use it to get performance data of each individual container. This insight clearly highlighted the importance of realistic architectural planning where In real word scenarios publisher devices would operate independently and not consume host resources the way Docker-based simulations do.

The system also successfully maintained end-to-end message delivery without data loss, even under the heaviest load. And real time visualization with Grafana proved to be a very useful tool for identifying issues and understanding system behavior dynamically.

Overall, the project achieved its objectives by providing a robust testbed for performance evaluation, yielding insights that are valuable not only for academic purposes but also for practical IoT deployment scenarios.

7. References

- [1] Prometheus Team, *Prometheus: Monitoring System & Time Series Database*. [Online]. Available: <https://prometheus.io/docs/introduction/overview/>
- [2] MetricFire, *How to Monitor Docker with Telegraf*, Mar. 11, 2025. [Online]. Available: <https://www.metricfire.com/blog/monitor-docker-performance-w/>
- [3] InfluxData, *Best Practices for Writing Data*. [Online]. Available: <https://docs.influxdata.com/influxdb/v2/write-data/best-practices/>
- [4] Grafana Labs, *Learn How to Monitor IoT Devices with Grafana*, Apr. 24, 2023. [Online]. Available: <https://grafana.com/blog/2023/04/24/learn-how-to-monitor-iot-devices-with-grafana/>
- [5] MQTT.org, *MQTT Version 3.1.1 Protocol Specification*. [Online]. Available: <https://mqtt.org/>
- [6] *Node Exporter and Prometheus Integration*. [Online]. Available: <https://prometheus.io/docs/guides/node-exporter/>
- [7] Docker Inc., *Docker Compose Documentation*. [Online]. Available: <https://docs.docker.com/compose/>